**Лабораторная работа № 2**

Цепочечные команды

Вариант 7

# 1 Цель работы

Изучить цепочечные команды и приобрести практические навыки их применения для обработки строк символов.

# 2 Постановка задачи

Написать программу на языке ассемблера, которая по исходной строке формирует палиндром, используя цепочечные команды.

Способ построения: к исходной строке $S$ дописывается её обратная запись $S^R$. Результат $P = S\,S^R$ одинаково читается в обоих направлениях. Например, `abcd` преобразуется в `abcddcba`.

Программа предназначена для 32-разрядной Windows и собирается MASM. Вводится строка длиной до 255 символов в однобайтовой кодировке CP866, включая русские буквы. Пробелы, регистр и знаки препинания сохраняются. Пустая строка даёт пустой результат; слишком длинный ввод отклоняется.

# 3 Алгоритм

1. Получить дескрипторы консоли и прочитать исходную строку функцией `ReadConsoleA`. Проверить длину и удалить из учитываемой длины завершающие символы CR и LF. Обозначить длину строки через $n$.

2. Сбросить флаг направления командой `CLD`. Поместить адреса исходного и результирующего буферов в `ESI` и `EDI`, а $n$ в `ECX`. Скопировать строку командой `REP MOVSB`.

3. Если $n = 0$, перейти к выводу. Иначе установить `ESI` на последний символ исходной строки и выполнить $n$ повторений: `STD; LODSB; CLD; STOSB`. `LODSB` читает символ в `AL` и уменьшает `ESI`, а `STOSB` записывает символ и увеличивает `EDI`.

4. Добавить CR и LF в конец результата. Вывести $2n + 2$ байта функцией `WriteConsoleA` и завершить программу.

Время работы составляет $O(n)$; буфер результата содержит $2n + 2$ байта с учётом перевода строки. Перед вызовами функций Windows флаг DF сброшен.

# 4 Использованные программные средства

- `.386`, `.MODEL FLAT, STDCALL` — набор команд процессора 80386, плоская модель памяти и соглашение вызовов STDCALL.
- `REP MOVSB` — копирование $n$ байт из `[ESI]` в `[EDI]`, счётчик повторений — `ECX`.
- `STD`, `CLD` — выбор направления движения указателей; `LODSB`, `STOSB` — чтение байта в `AL` и запись байта из `AL`.
- `GetStdHandle`, `ReadConsoleA`, `WriteConsoleA` — получение дескрипторов, ввод и вывод; `ExitProcess` — завершение.
- `SetConsoleCP`, `SetConsoleOutputCP` — установка CP866; `PrintText` — вывод заданного количества байтов.

# 5 Тестирование

Программа собрана ассемблером MASM и компоновщиком Microsoft Link для консольного приложения Windows x86. Проверено 15 запусков исполняемого файла в консоли Windows. В каждом успешном тесте фактический результат совпал с ожидаемым, имел длину $2n$ и совпадал со своей обратной записью.

В таблице $\varepsilon$ обозначает пустую строку, ␣ — пробел, а степень — число повторений строки.

| № | Исходная строка | Фактический результат | Код выхода |
|---|---|---|---|
| 1 | $\varepsilon$ | $\varepsilon$ | 0 |
| 2 | `a` | `aa` | 0 |
| 3 | `ab` | `abba` | 0 |
| 4 | `abc` | `abccba` | 0 |
| 5 | `abcd` | `abcddcba` | 0 |
| 6 | `aba` | `abaaba` | 0 |
| 7 | `aaaa` | `aaaaaaaa` | 0 |
| 8 | `a b` | `a bb a` | 0 |
| 9 | ␣$a$␣ | ␣$a$␣␣␣$a$␣ | 0 |
| 10 | `Ab1!` | `Ab1!!1bA` | 0 |
| 11 | `тест` | `тесттсет` | 0 |
| 12 | `а роза` | `а розаазор а` | 0 |
| 13 | (`abc`)$^{85}$, 255 символов | (`abc`)$^{85}$(`cba`)$^{85}$, 510 символов | 0 |
| 14 | `a`$^{256}$ | Ошибка: превышена длина ввода | 1 |
| 15 | `b`$^{300}$ | Ошибка: превышена длина ввода | 1 |

**Протоколы отдельных запусков**
Ниже приведён фактический текст консоли, сохранённый при тестировании. Полные результаты всех 15 проверок находятся в `code/tests.txt`.

**Тест № 5**
Назначение: формирование палиндрома из строки чётной длины.
Ожидается: `abcddcba`.

```
Enter a string (max 255 characters): abcd
Palindrome: abcddcba
```

**Тест № 1**
Назначение: проверка пустой исходной строки.
Ожидается: пустой результат без ошибки.

```
Enter a string (max 255 characters):
Palindrome:
```

**Тест № 8**
Назначение: проверка сохранения пробелов.
Ожидается: `a bb a`.

```
Enter a string (max 255 characters): a b
Palindrome: a bb a
```

# 6 Код программы

```
.386
.MODEL FLAT, STDCALL
.STACK 4096

EXTERN GetStdHandle@4:PROC
EXTERN SetConsoleCP@4:PROC
EXTERN SetConsoleOutputCP@4:PROC
EXTERN ReadConsoleA@20:PROC
EXTERN WriteConsoleA@20:PROC
EXTERN ExitProcess@4:PROC

MAX_LENGTH EQU 255

.DATA
prompt       DB 'Enter a string (max 255 characters): '
promptLength EQU $ - prompt
resultText   DB 'Palindrome: '
resultLength EQU $ - resultText
errorText    DB 'Error: input failed or string is longer than 255 characters.',13,10
errorLength  EQU $ - errorText
inputHandle  DD ?
outputHandle DD ?
readCount    DD ?
written      DD ?
stringLength DD ?
inputBuffer  DB MAX_LENGTH + 3 DUP (?)
outputBuffer DB 2 * MAX_LENGTH + 2 DUP (?)

.CODE
; Print EDX bytes starting at EAX to the console.
PrintText PROC
```

```
    PUSH 0
    PUSH OFFSET written
    PUSH EDX
    PUSH EAX
    PUSH outputHandle
    CALL WriteConsoleA@20
    RET
PrintText ENDP

MAIN PROC
    ; CP866 stores each supported character, including Russian letters, in one byte.
    PUSH 866
    CALL SetConsoleCP@4
    PUSH 866
    CALL SetConsoleOutputCP@4

    PUSH -10
    CALL GetStdHandle@4
    MOV inputHandle, EAX
    PUSH -11
    CALL GetStdHandle@4
    MOV outputHandle, EAX

    MOV EAX, OFFSET prompt
    MOV EDX, promptLength
    CALL PrintText

    PUSH 0
    PUSH OFFSET readCount
    PUSH SIZEOF inputBuffer
    PUSH OFFSET inputBuffer
    PUSH inputHandle
    CALL ReadConsoleA@20
    TEST EAX, EAX
    JZ inputError

    ; The console adds CR and LF. They are not part of the source string.
    MOV ECX, readCount
    CMP ECX, 2
    JB inputError
    SUB ECX, 2
    CMP ECX, MAX_LENGTH
    JA inputError
    CMP BYTE PTR inputBuffer[ECX], 13
    JNE inputError
    CMP BYTE PTR inputBuffer[ECX+1], 10
    JNE inputError
    MOV stringLength, ECX

    ; Copy the source string to the first half of the result.
    CLD
    MOV ESI, OFFSET inputBuffer
    MOV EDI, OFFSET outputBuffer
    REP MOVSB

    ; Append the source string in reverse order.
    MOV ECX, stringLength
    JECXZ palindromeReady
    LEA ESI, inputBuffer[ECX-1]
copyBackwards:
    STD
    LODSB
    CLD
    STOSB
    LOOP copyBackwards
```

```
palindromeReady:
    ; DF is zero before any Windows API call.
    MOV AL, 13
    STOSB
    MOV AL, 10
    STOSB

    MOV EAX, OFFSET resultText
    MOV EDX, resultLength
    CALL PrintText

    MOV EAX, OFFSET outputBuffer
    MOV EDX, stringLength
    ADD EDX, EDX
    ADD EDX, 2
    CALL PrintText

    PUSH 0
    CALL ExitProcess@4

inputError:
    MOV EAX, OFFSET errorText
    MOV EDX, errorLength
    CALL PrintText
    PUSH 1
    CALL ExitProcess@4
MAIN ENDP
END MAIN
```

# 7 Вывод

Разработана программа формирования палиндрома с использованием `REP MOVSB`, `LODSB` и `STOSB`. Правильность работы подтверждена обычными и граничными тестами.